UNIWERSYTET
IM. ADAMA MICKIEWICZA
W POZNANIU

Wydział Matematyki i Informatyki

Dominik Jagosz
Numer albumu: 464871

Escape From Morasko - gra mobilna typu rogue-like: Tworzenie oprogramowania z użyciem języka Kotlin

Escape From Morasko - rogue-like mobile game: Using the Kotlin programming language for software development

Praca inżynierska na kierunku **informatyka**
napisana pod opieką
dra Wojciecha Wawrzyniaka

Poznań, grudzień 2023

Poznań, 2 lutego 2024 r.

Oświadczenie

Ja, niżej podpisany **Dominik Jagosz**, student Wydziału Matematyki i Informatyki Uniwersytetu im. Adama Mickiewicza w Poznaniu oświadczam, że przedkładaną pracę dyplomową pt. *Escape From Morasko - gra mobilna typu rogue-like: Tworzenie oprogramowania z użyciem języka Kotlin* napisałem samodzielnie. Oznacza to, że przy pisaniu pracy, poza niezbędnymi konsultacjami, nie korzystałem z pomocy innych osób, a w szczególności nie zlecałem opracowania rozprawy lub jej części innym osobom, ani nie odpisywałem tej rozprawy lub jej części od innych osób. Oświadczam również, że egzemplarz pracy dyplomowej w wersji drukowanej jest całkowicie zgodny z egzemplarzem pracy dyplomowej w wersji elektronicznej. Jednocześnie przyjmuję do wiadomości, że przypisanie sobie, w pracy dyplomowej, autorstwa istotnego fragmentu lub innych elementów cudzego utworu lub ustalenia naukowego stanowi podstawę stwierdzenia nieważności postępowania w sprawie nadania tytułu zawodowego.

[TAK] wyrażam zgodę na udostępnianie mojej pracy w czytelni Archiwum UAM

[TAK] wyrażam zgodę na udostępnianie mojej pracy w zakresie koniecznym do ochrony mojego prawa do autorstwa lub praw osób trzecich

Streszczenie

Kotlin to wysokopoziomowy, wieloparadygmatowy język programowania, działający na maszynie wirtualnej Javy. W pracy opisana będzie jego specyfika, głownie w podejściu obiektowym. Przedstawione zostaną funkcjonalności najbardziej użyteczne w procesie wytwarzania oprogramowania na przykładzie gry mobilnej *Escape from Morasko*. Szczególna uwaga będzie skupiona na elementach wyróżniających Kotlin na tle języka Java i ułatwiających tworzenie kodu źródłowego.

Słowa kluczowe: Kotlin, JVM, libGDX

Abstract

The Kotlin programming language is a high-level, multi-paradigm programming language, targeting the Java Virtual Machine. This thesis presents its use mainly in object-oriented programming. Its most useful functionalities in software development will be described based on the example of the mobile game *Escape from Morasko*. Special attention will be focused on features that differentiate Kotlin from the Java programming language and help in the process of creating source code.

Keywords: Kotlin, JVM, libGDX

Spis treści

Rozdział 1. Tworzenie oprogramowania z użyciem języka Kotlin 6
1.1. Wstęp 6
1.1.1. Escape from Morasko 6
1.2. Kotlin jako język działający na maszynie wirtualnej Javy 6
1.2.1. JVM - maszyna wirtualna Javy 6
1.2.2. JVM jako maszyna Turinga 7
1.2.3. Kod bajtowy Javy 7
1.2.4. Kompilacja do kodu bajtowego Javy 9
1.2.5. Wykonywanie kodu 9
1.2.6. Wykonywanie kodu źródłowego w języku Kotlin 10
1.2.7. Przykład 12
1.2.8. Integracja Kotlina z libGDX 15
1.3. Programowanie obiektowe 15
1.3.1. Widoczność 15
1.3.2. Konstruktor 16
1.3.3. Singleton, Object i pola statyczne 18
1.3.4. Przykład 21
1.4. Klonowanie obiektów 24
1.4.1. Motywacja 24
1.4.2. Cloneable 25
1.4.3. Data class 25
1.5. Zarządzanie wartościami pustymi 27
1.5.1. Null 27
1.5.2. Bezpieczne używanie zmiennych nullowalnych 27
1.6. Gotowe typy danych i funkcje wykorzystywane w projekcie 28
1.6.1. Kolekcje 28
1.6.2. Funkcje rozszerzające 29
1.7. Podsumowanie 29
1.7.1. Korzyści zastosowania języka Kotlin 29

Rozdział 1

Tworzenie oprogramowania z użyciem języka Kotlin

1.1. Wstęp

1.1.1. Escape from Morasko

Escape from Morasko to taktyczna gra turowa z gatunku rogue-like, przeznaczona na urządzenia mobilne z systemem Android. Została zaprojektowana w oparciu o wszechstronne narzędzie do tworzenia gier libGDX, napisane w języku programowania Java. Korzystanie z libGDX może odbywać się z zastosowaniem dowolnego języka działającego na maszynie wirtualnej Javy. W celu realizacji *Escape from Morasko* zdecydowano się na użycie języka Kotlin. Kotlin to otwartoźródłowy, wielofunkcyjny język programowania, rozwijany od 2011 r. przez spółkę JetBrains. Powstał jako nowoczesna alternatywa dla Javy. W 2019 r. zastąpił ją jako preferowany przez Google język do tworzenia aplikacji dla systemu Android.

1.2. Kotlin jako język działający na maszynie wirtualnej Javy

1.2.1. JVM - maszyna wirtualna Javy

Maszyna wirtualna to abstrakcja fizycznego komputera, tworząca środowisko, w którym wykonywane są programy. Może ona odwzorowywać zachowanie konkretnych komponentów komputerowych albo działać niezależnie od specyfi-

kacji sprzętowej. Niektóre maszyny wirtualne mogą kompletnie odwzorowywać działanie komputera, emulując zachowanie systemu operacyjnego i obsługując wiele procesów. Natomiast maszyna wirtualna Javy służy do wykonywania pojedynczych, specjalnie przygotowanych dla niej aplikacji. Korzystanie z JVM likwiduje konieczność przystosowywania programów do działania na konkretnej platformie sprzętowej[1]. Zamiast tego umożliwia stworzenie jednej aplikacji, która może być uruchomiona na każdej maszynie, na której działa JVM. Upraszcza to proces przygotowywania wieloplatformowego oprogramowania.

1.2.2. JVM jako maszyna Turinga

Maszyna wirtualna Javy jest zupełna w sensie Turinga. Jej rozkazami są kody operacji (ang. *operation codes*, skracane w specyfikacji JVM do *opcodes*). Każdy kod operacji jest zapiany na jednym bajcie. Argumenty operacji, czyli obiekty, na których wykonywane są działania, nazywane są operandami. JVM działa zarówno na stosie, jak i na rejestrach - stos operandów służy do wykonywania operacji, a w rejestrach zapisywane są zmienne lokalne. Zbiór wszystkich kodów operacji możliwych do wykonania na JVM nazywa się kodem bajtowym Javy.

1.2.3. Kod bajtowy Javy

Kody operacji i typy danych

Kody operacji definiują instrukcje, jakie będą wykonane na operandach. Sposób wykonywania tych instrukcji może różnić się w zależności od typu jej argumentów. Nazwy kodów operacji składają się w większości z sufiksu określającego wykonywaną instrukcję, oraz prefiksu wyznaczającego typ danych, na których operuje instrukcja. Typy danych używane przez JVM są przedstawione na rysunku 1.1.

Kategorie instrukcji

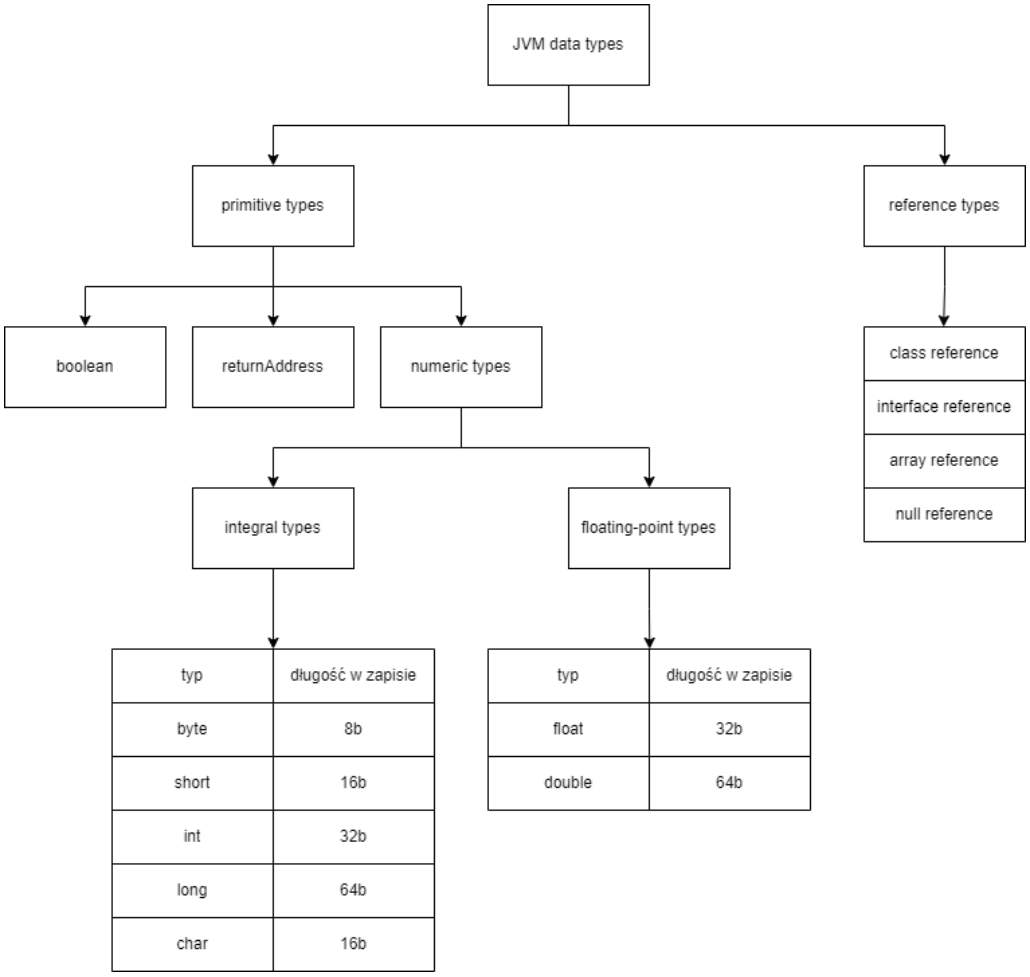
Instrukcje można skategoryzować jako:

— **ładujące i zapisujące** - przenoszą dane pomiędzy stosem operandów i zmiennymi lokalnymi

— **arytmetyczne i logiczne** - wykonują działania arytmetyczne i działania na bitach

— **zmieniające typ danych** - dokonują konwersji pomiędzy typami numerycznymi (1.1 *numeric types*)

[1] Rozumianej jako zbiór komputerów o spójnej architekturze, używających tego samego kodu maszynowego.



Rysunek 1.1. Typy danych używane przez JVM.
Opracowanie na podstawie [3].

— **tworzące obiekty i manipulujące nimi** - tworzą instancje klas i tablic, mają dostęp do pól obiektów i komórek tablic
— **manipulujące stosem operandów**
— **przenoszące sterowanie** - zmieniają kolejność wykonywania instrukcji, mogą to robić warunkowo (*if*) lub bezwarunkowo (*goto*)
— **wywołujące metody obiektów i wracające z nich**

Format pliku .class

Kod bajtowy Javy jest niezależny od języka Java. JVM może obsługiwać aplikacje napisane we wszystkich językach programowania, których kod źródłowy może być kompilowany do plików o rozszerzeniu `.class`. Plik `.class` zawiera skompilowany kod bajtowy Javy i stanowi opis pewnej klasy. Jego format określa specyfikacja JVM. Przykłady plików `.class` są przedstawione na wyciągach 1.1 - 1.4.

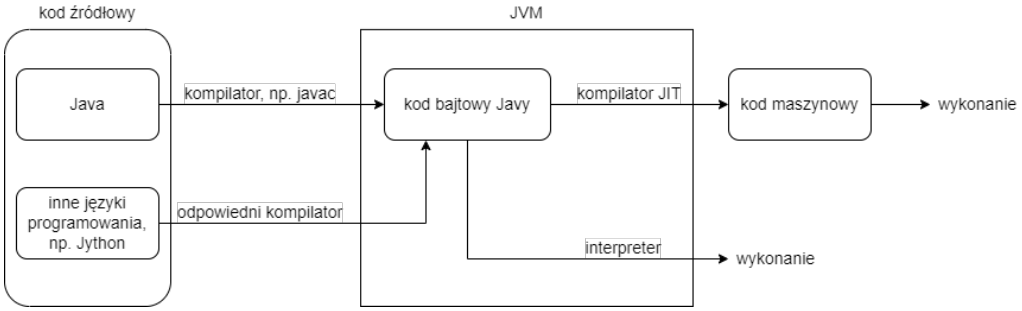
1.2.4. Kompilacja do kodu bajtowego Javy

Mimo że jest to teoretycznie możliwe, oprogramowania nie pisze się w kodzie bajtowym. Do tego służą języki programowania, które mają o wiele czytelniejszą składnię, są łatwiejsze do zrozumienia przez człowieka i operują na wyższym poziomie abstrakcji. Opis działania programu napisany w pewnym języku programowania nazywa się kodem źródłowym. W celu wykonania kod źródłowy musi być przetłumaczony na sekwencję instrukcji procesora komputera, na którym ma być uruchomiony. Różne sposoby przeprowadzania tej zmiany są przedstawione na rysunku 1.2. Języki przeznaczone na JVM, takie jak Java i Kotlin, posiadają kompilatory tłumaczące je na język bajtowy Javy. Inne języki programowania bywają przystosowywane do kompilacji na JVM, np. Jython, implementacja języka Python napisana w Javie. Kod bajtowy Javy, będący wynikiem kompilacji kodu źródłowego, jest dalej przetwarzany przez JVM, która może wykonywać go linijka po linijce, używając interpretera, albo skompilować go do kodu maszynowego, wykorzystując kompilator JIT.

1.2.5. Wykonywanie kodu

Kompilator JIT

Kompilator JIT (ang. *Just-in-time*) dokonuje kompilacji fragmentami wtedy, kiedy potrzeby jest dany fragment. Bezpośrednio po uruchomieniu JVM wywoływanych jest bardzo wiele metod, a ich kompilacja obciąża procesor i pamięć operacyjną. Skutkuje to znaczącym zwiększeniem czasu potrzebnego na uru-



Rysunek 1.2. Ścieżki kompilacji.

chomienie aplikacji, w porównaniu z wykorzystywaniem interpretera. Żeby go skrócić, metody są kompilowane, dopiero gdy liczba ich wywołań przekroczy pewną określoną liczbę. Do tego czasu są interpretowane. Często używane metody mogą być kompilowane ponownie, w celu lepszej ich optymalizacji.

Kompilator AOT

Tradycyjny kompilator AOT (ang. *Ahead-of-time*) zmienia cały kod aplikacji w kod maszynowy przed uruchomieniem. Jest to długotrwały proces, dlatego bywa wykonywany poza maszyną, na której docelowo ma być uruchomiony program. W takim przypadku dostarczane są na nią tylko pliki wykonywalne, będące wynikiem kompilacji. Zapewnia to szybki czas uruchamiania. Z kolei prędkość działania w porównaniu z wykorzystywaniem kompilatora JIT zależy od aplikacji, ponieważ oba kompilatory mają do dyspozycji odmienne techniki optymalizacji.

Uproszczone porównanie metod wykonywania kodu zawiera tabela 1.1. Z uwagi na różnice między nimi znajdują one zastosowanie w odmiennych sytuacjach, np. podczas wytwarzania i debugowania aplikacji pisanych w języku Dart używa się kompilatora JIT, a do przygotowania ich do dystrybucji - kompilatora AOT.

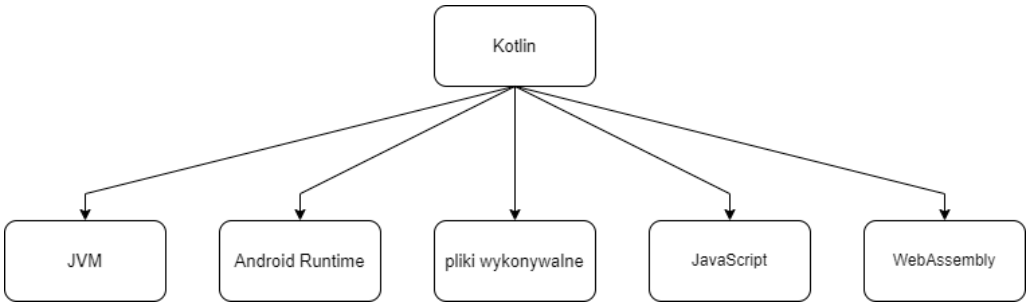
1.2.6. Wykonywanie kodu źródłowego w języku Kotlin

Kotlin, oprócz możliwości kompilacji do kodu bajtowego Javy i uruchamiania na JVM, oraz - w przypadku aplikacji przeznaczonych na system Android - działania w środowisku uruchomieniowym *Android Runtime* (*ART*)[2], oferuje również inne sposoby wykonywania. Dostępny kompilator AOT tworzy pliki wykonywalne zawierające kod maszynowy, m.in. platform firmy Apple. Umoż-

[2] Które używa podobnej maszyny wirtualnej i kodu bajtowego

Tabela 1.1. Sposoby wykonywania kodu

kompilator AOT	konieczność wcześniejszej kompilacji	szybkie uruchamianie i działanie
kompilator JIT	powolne uruchamianie	szybkie, optymalizowane na bieżąco działanie
interpreter	szybkie uruchamianie	powolne działanie



Rysunek 1.3. Kompilacja kodu źródłowego języka Kotlin.

liwia to wytwarzanie aplikacji wieloplatformowych działających na systemach iOS, na których nie jest wspierana JVM. Kotlin może być również tłumaczony na język JavaScript i działać w przeglądarce lub na *node.js*. Rozwijana jest eksperymentalna opcja kompilacji do Web Assembly i działania razem z JavaScript.

Przykład zawierający kod źródłowy w języku Kotlin i jego odpowiednik po kompilacji do kodu bajtowego Javy jest przedstawiony na wyciągach 1.1 - 1.4. Warto wspomnieć, że dla celów optymalizacji, przed zmianą w docelowy kod, kod źródłowy napisany w Kotlinie jest przekształcany w kod pośredni (*Kotlin intermediate representation*).

1.2.7. Przykład

Kod źródłowy napisany w języku Java - przykładowa metoda `test()` klasy `Klasa`, zawarta w pliku `Klasa.java`.

wyciag 1.1. `Klasa.java`

```
public final class Klasa {
    int test() {
        int a = 2;
        int b = 2;
        return a + b;
    }
}
```

Kod źródłowy w języku Kotlin analogicznej funkcji `test()`, zawarty w pliku `Plik.kt`. Kotlin umożliwia pisanie funkcji, które nie są metodami istniejącej klasy.

wyciag 1.2. `Plik.kt`

```
fun test(): Int {
    val a = 2
    val b = 2
    return a + b
}
```

Fragment pliku `Klasa.class`, zwierającego kod bajtowy Javy otrzymany w wyniku kompilacji pliku `Klasa.java`. Zwracają uwagę instrukcje odpowiadają-

ce za umieszczanie operandów na stosie i dodających dane typu `int`. Można odczytać maksymalny rozmiar stosu operandów i zmienne lokalne. Podgląd kodu bajtowego umożliwia plugin *Bytecode Viever*. Alternatywnie można użyć polecenia `javap -c -v Klasa.class` w katalogu zawierającym pliki wynikowe.

wyciag 1.3. `Klasa.class`

```
public class Klasa {
  // compiled from: Klasa.java
  ...
  // access flags 0x0
  test()I
   L0
    LINENUMBER 3 L0
    ICONST_2
    ISTORE 1
   L1
    LINENUMBER 4 L1
    ICONST_2
    ISTORE 2
   L2
    LINENUMBER 5 L2
    ILOAD 1
    ILOAD 2
    IADD
    IRETURN
   L3
    LOCALVARIABLE this LKlasa; L0 L3 0
    LOCALVARIABLE a I L1 L3 1
    LOCALVARIABLE b I L2 L3 2
    MAXSTACK = 2
    MAXLOCALS = 3
}
```

Fragment pliku `PlikKt.class`, zwierającego kod bajtowy Javy otrzymany w wyniku kompilacji pliku `Plik.kt`. Funkcja `test()` stała się metodą klasy `PlikKt`. Domyślnie w Kotlinie klasy są *final* (nie można po nich dziedziczyć).

Warto zauważyć, że metoda `test()` nie posiada zmiennej lokalnej `this`, obecnej w pliku `Klasa.class`.

wyciag 1.4. `PlikKt.class`

```
public final class PlikKt {
  // access flags 0x19
  public final static test()I
   L0
    LINENUMBER 2 L0
    ICONST_2
    ISTORE 0
   L1
    LINENUMBER 3 L1
    ICONST_2
    ISTORE 1
   L2
    LINENUMBER 4 L2
    ILOAD 0
    ILOAD 1
    IADD
    IRETURN
   L3
    LOCALVARIABLE b I L2 L3 1
    LOCALVARIABLE a I L1 L3 0
    MAXSTACK = 2
    MAXLOCALS = 2
  ...
  // compiled from: Plik.kt
}
```

W tym prostym przykładzie kod źródłowy dwóch funkcji wykonujących to samo zadanie, napisany w różnych językach, został przekształcony w praktycznie identyczny kod bajtowy Javy. W bardziej skomplikowanych przypadkach mogą pojawiać się różnice, w zależności od wybranego kompilatora. W przykładzie zastosowano najpopularniejszy kompilator Javy javac, będący częścią OpenJDK. Kompilator Kotlina jest rozwijany przez twórców języka. Obecnie trwają prace nad jego nową i ulepszoną wersją, nazwaną *K2*. 15 grudnia 2023 udostępniono testową wersję `2.0.0-Beta2`, w której *K2* jest stosowany domyślnie.

1.2.8. Integracja Kotlina z libGDX

Gra *Escape from Morasko* została stworzona z wykorzystaniem wszechstronnego narzędzia do tworzenia gier libGDX. LibGDX jest napisany w języku Java i C/C++, a używające go aplikacje są kompilowane do kodu bajtowego Javy, który może być uruchamiany na komputerach i telefonach z systemem Android. Działanie na systemach iOS, na których nie działa JVM, jest zapewniane przez *MobiVM* - narzędzie pozwalające kompilować kod bajtowy Javy do kodu maszynowego platform Apple. Stąd możliwe jest używanie libGDX z każdym językiem przeznaczonym na JVM, w którym można wywoływać funkcje napisane w języku Java, w szczególności użytego w *Escape from Morasko* języka Kotlin.

LibGDX może też służyć do tworzenia gier działających w przeglądarkach. Polecane jest wówczas zastosowanie narzędzia GWT (*Google Web Toolkit*), które przekształca kod Javy na kod JavaScript. Wyklucza to używanie innych języków przeznaczonych na JVM niż Java. Problem ten mógłby być rozwiązany poprzez wykorzystanie narzędzia *TeaVM*, które zamiast kodu źródłowego tłumaczy kod bajtowy Javy. Inną potencjalną opcją jest skorzystanie z zawartej w kompilatorze Kotlina możliwości kompilacji do JavaScriptu. Wykracza to jednak poza zakres tej pracy.

1.3. Programowanie obiektowe

1.3.1. Widoczność

Jak wcześniej wspomniano, Kotlin umożliwia tworzenie funkcji bądź zmiennych, które nie są metodami ani polami żadnej klasy. Takie obiekty są domyślnie widoczne publicznie, a więc wszędzie w pliku, gdzie deklarowany jest obiekt, i w każdym miejscu, które uzyska do niego dostęp przez importowanie. Tak zachowują się obiekty deklarowane bez żadnego poprzedzającego słowa kluczowego bądź ze słowem `public` (publiczne). Użycie słowa `private` (prywatne) ogranicza widoczność do jednego pliku. Kotlin nie łączy pakietów z katalogami systemu plików tak silnie jak Java. Możliwa jest kompilacja pliku o przypisanej nazwie pakietu, która nie odpowiada nazwie katalogu, w którym znajduje się plik. Stąd nie można ograniczyć widoczności obiektu do jednego pakietu, co jest domyślnym zachowaniem w Javie, a nazywane jest określeniem `package-private` (prywatne w obrębie pakietu). Zamiast tego Kotlin używa słowa kluczowego `internal` (wewnętrzne), które ogranicza widoczność do danego modułu. Moduł rozumiany jest jako zbiór plików zawierających kod

źródłowy, które są kompilowane razem, np. skonfigurowane przez narzędzie *Gradle*, jak w przypadku *Escape from Morasko*.

Pola i metody klas mogą być poprzedzone jednym z wymienionych dotychczas słów kluczowych, jak również dostępnym tylko dla nich modyfikatorem widoczności `protected` (chronione). W odróżnieniu od pola prywatnego, widocznego wyłącznie w obrębie klasy, w której jest deklarowane, pole chronione jest dostępne również w klasach pochodnych. Warto wspomnieć, że klasy pochodne, nadpisując pola i metody klasy, nie mogą zmniejszać poziomu ich widoczności.

1.3.2. Konstruktor

Każda klasa deklarowana przy użyciu słowa kluczowego `class` posiada co najmniej jeden konstruktor służący do tworzenia obiektów. Podobnie jak w Javie, nosi on taką samą nazwę jak klasa, jednak jego deklaracja wymaga słowa `constructor`.

wyciag 1.5. `Konstruktor w języku Java`

```
public class Klasa {
    public Klasa() {
        System.out.println("Hello");
    }
    private Klasa(String parametr) {
        System.out.println("Hello");
        System.out.println(parametr);
    }
    public void test() {
        Klasa o1 = new Klasa();
        Klasa o2 = new Klasa("World!");
    }
}
```

wyciag 1.6. `Konstruktor w języku Kotlin`

```
class Klasa {
    constructor() {
        println("Hello")
    }
```

```
    private constructor(parametr: String) {
        println("Hello")
        println(parametr)
    }
    fun test() {
        val o1 = Klasa()
        val o2 = Klasa("World!")
    }
}
```

Konstruktory znajdujące się wewnątrz klasy, których może być wiele, to *secondary constructors* (konstruktory podrzędne). Jeden *primary constructor* (konstruktor podstawowy) może być deklarowany w nagłówku klasy. Jeżeli istnieje, wszystkie konstruktory podrzędne muszą korzystać albo z niego, albo z innego konstruktora podrzędnego. Oznacza się to przez dodanie wyrażenia `: this()`, z odpowiednimi parametrami w nawiasach, przed ciałem konstruktora. Kod konstruktora podstawowego jest umieszczany w blokach `init{}`, które mogą być przeplatane deklaracjami zmiennych, i jest wykonywany przy każdym wywołaniu konstruktora podrzędnego. Na poniższym wyciągu przedstawiona jest klasa z konstruktorem podstawowym, w której funkcja testowa daje rezultat tożsamy z poprzednimi.

wyciag 1.7. `Konstruktor podstawowy z polem init{}`

```
class Klasa() {
    init {
        println("Hello")
    }
    private constructor(parametr: String) : this() {
        println(parametr)
    }
    fun test() {
        val o1 = Klasa()
        val o2 = Klasa("World!")
    }
}
```

Jeżeli nie zdefiniowano żadnego konstruktora, zostanie wygenerowany domyślny, nie posiadający parametrów. Obecność takiego konstruktora może być

wymagana podczas pracy z niektórymi narzędziami programistycznymi. W *Escape from Morsko* wszystkie klasy implementujące interfejs `Json.Serializable` muszą mieć bezparametrowy konstruktor. Jest on używany podczas tworzenia obiektów ładowanych z plików, zawierających zapisany stan gry. Uzyskiwano go, dodając konstruktor podrzędny, który wywoływał konstruktor podstawowy z pewnymi wartościami, lub dodając domyślne wartości bezpośrednio w konstruktorze podstawowym, co wyraża się przez użycie znaku równości, jak w przykładzie poniżej. Przedstawia on klasę z prywatnym konstruktorem podstawowym, którego parametr ma podaną wartość domyślną. Żeby zmienić widoczność konstruktora podstawowego, trzeba dodać słowo `constructor`.

wyciag 1.8. `Prywatny konstruktor podstawowy z parametrem o wartości domyślnej`

```
class Klasa private constructor(parametr: Int = 0) {
    init {
        println(parametr)
    }
    fun test() {
        val o1 = Klasa()
        val o2 = Klasa(10)
    }
}
```

1.3.3. Singleton, Object i pola statyczne

Singleton

Prywatny konstruktor znajduje zastosowanie w implementacji wzorca Singleton. Jest wywoływany tylko raz, wewnątrz klasy, a stworzony obiekt jest udostępniany przez statyczną metodę. Poniżej przedstawiona jest tradycyjna implementacja w Javie.

wyciag 1.9. `Singleton w języku Java (lazy initialization)`

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {
    }
```

```
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Przykład ten ilustruje tzw. wersję leniwą (ang. *lazy initialization*) - obiekt jest tworzony w momencie pierwszego użycia metody `getInstance()`, a nie podczas ładowania klasy z pliku `.class` na JVM. Powyższy kod niekoniecznie zadziała w oczekiwany sposób w programie wielowątkowym. Możliwe, że w dwóch wątkach jednocześnie nastąpi wywołanie `getInstance()` i stworzone zostaną dwa obiekty klasy Singleton. Problem ten można rozwiązać przez dodanie wzorca *double-checked locking* lub *initialization-on-demand holder*. Wprowadzają one jednak dużo skomplikowanego kodu, a błędnie wdrożone mogą powodować komplikacje. Proste i bezpieczne w aplikacjach wielowątkowych rozwiązanie wymaga tworzenia obiektu w trakcie ładowania klasy (ang. *eager initialization*). W Javie można je wykonać przy deklaracji zmiennej albo w bloku `static`, gdzie przypisywane są wartości zmiennym statycznym. Ładowanie klasy Singleton i tak nastąpi dopiero gdy po raz pierwszy zostanie ona użyta. W większości przypadków stanie się to w momencie wywołania `getInstance()`, czyli tak samo jak w wersji leniwej.

wyciag 1.10. `Singleton w języku Java (`*`eager initialization`*`)`

```
public class Singleton {
    private static Singleton instance = new Singleton();
    private Singleton() {
    }
    public static Singleton getInstance() {
        return instance;
    }
}
```

Object

Z uwagi na opisane trudności w prawidłowej implementacji wzorca Singleton, mając na celu uproszczenie i ujednolicenie kodu, w języku Kotlin zaprojek-

towano deklarację `object`. Object to klasa tworzona tylko raz, w momencie pierwszego użycia. Jest porównywalna do Singletona tworzonego podczas ładowania, jest bezpieczna w użyciu w aplikacjach wielowątkowych. Nie można używać konstruktora object, a dostęp do pól i metod odbywa się przez użycie jego nazwy. Wykorzystywana składnia przypomina sposób użycia klasy statycznej w języku Java. Kotlin nie udostępnia możliwości tworzenia statycznych klas ani statycznych pól i metod w zwykłych klasach. Zamiast dodawać klasy statyczne, można deklarować zmienne i funkcje bezpośrednio w pliku z kodem źródłowym. Natomiast logikę, która byłaby wyrażona jako statyczne pola i metody klasy, należy umieścić w zawartej w niej deklaracji object.

Companion object

Deklaracja object zawarta w klasie może być poprzedzona słowem kluczowym `companion`. Tak zdefiniowany obiekt towarzyszący klasie jest ładowany razem z nią. Dostęp do zawartych w nim pól i metod odbywa się przez użycie nazwy zawierającej go klasy. Jest to działanie analogiczne do zachowania pól i metod statycznych w języku Java. Możliwe jest nawet wymuszenie kompilacji pól obiektu towarzyszącego do pól statycznych na JVM przez poprzedzenie ich adnotacją `@JvmStatic`. Domyślnie jednak będą one reprezentowane jako zwykłe pola pewnego obiektu.

Przypisania object

Poza deklarowaniem object jako klasy wewnętrznej, możliwe jest również przypisywanie go do zmiennej. Bywa to użyteczne z uwagi na fakt, że object może implementować interfejs lub dziedziczyć po pewnej klasie. Wtedy zmienna, do której object zostaje przypisany, jest typu implementowanego interfejsu lub nadklasy.[3]

Poniżej przedstawione są trzy sposoby używania słowa kluczowego `object`. Obiekt towarzyszący klasie zostanie stworzony podczas jej ładowania. Można nie definiować jego nazwy, wtedy dostaje domyślną nazwę `Companion`. Może być wywoływany przez swoją nazwę lub przez nazwę klasy. Obiekt `Nazwany` zostanie stworzony podczas pierwszego użycia. Podobnie jak obiekt towarzyszący, jest dostępny tylko przez nazwę klasy, a nie przez nazwę obiektu. Z kolei obiekt przypisany do pola klasy jest tworzony w momencie przypisania, w tym przypadku podczas tworzenia obiektów klasy zawierającej pole. Dostęp do pola jest możliwy przez stworzony obiekt.

[3] Jeżeli object nie implementuje żadnego interfejsu, zmienna pozostaje typu `Any`. `Any` to klasa abstrakcyjna, która jest nadklasą każdej klasy w języku Kotlin.

wyciag 1.11. Deklaracje i przypisania object

```
interface Testowy {
    fun test(): String
}

class Klasa {
    companion object : Testowy {
        override fun test() = "companion object"
    }
    object Nazwany : Testowy {
        override fun test() = "named object"
    }
    val przypisany: Testowy = object : Testowy {
        override fun test() = "object expression"
    }
}

fun main() {
    println(Klasa.test())
    println(Klasa.Companion.test())
    println(Klasa.Nazwany.test())
    println(Klasa().przypisany.test())
}
```

1.3.4. Przykład

Escape from Morasko było tworzone z wykorzystaniem podejścia obiektowego, którego zasady, dzięki specyfice języka Kotlin, zostały rozluźnione. Język Java wymaga, aby każda publiczna klasa była umieszczona w oddzielnym pliku, a każda zmienna i funkcja należała do pewnej klasy. Programując w Kotlinie unika się tych ograniczeń, przez co możliwe jest umieszczanie powiązanych klas, zmiennych i funkcji w jednym pliku. Efektem takiej zmiany podejścia jest mniej rozbudowana hierarchia plików i łatwiejsze przeglądanie kodu, jeżeli zawierające go pliki liczą niewiele linii. W przypadku kiedy pliki są duże, dla

zwiększenia czytelności warto rozbić je, rozdzielając klasy. Małym plikiem jest na przykład `EnemyBatCorpse.kt`.

wyciag 1.12. `EnemyBatCorpse.kt wersja 1`

```
package com.efm.entities.enemies.bat

import ...

class EnemyBatCorpse(
        position : RoomPosition = RoomPosition(),
        loot : PossibleItems? = null,
        seed : Int = Random.nextInt()
                    ) : EnemyCorpse(position, loot, seed)
{
    override fun getTile() : TiledMapTile = Tiles.batCorpse
    override fun getOutlineYellowTile(n : Int) : TiledMapTile =
        Tiles.batCorpseOutlineYellow
    override fun getOutlineTealTile() : TiledMapTile? = null
}

val defaultLoot = PossibleItems()
```

Poza klasą opisującą zwłoki nietoperza, zawiera ona stałą przechowującą domyślny, to znaczy nie modyfikowany przez historię i obecny stan gry, łup możliwy do znalezienia w zwłokach. Jest publicznie dostępna, żeby można było używać jej w miejscach, w których tworzone są obiekty klasy `EnemyBatCorpse`. Innym sposobem jej udostępnienia jest umieszczenie jej w obiekcie towarzyszącym klasie, przedstawionym poniżej.

wyciag 1.13. `EnemyBatCorpse.kt wersja 2`

```
class EnemyBatCorpse(
        position : RoomPosition = RoomPosition(),
        loot : PossibleItems? = null,
        seed : Int = Random.nextInt()
                    ) : EnemyCorpse(position, loot, seed)
{
    override fun getTile() : TiledMapTile = Tiles.batCorpse
```

```
    override fun getOutlineYellowTile(n : Int) : TiledMapTile =
        Tiles.batCorpseOutlineYellow
    override fun getOutlineTealTile() : TiledMapTile? = null

    companion object
    {
        val defaultLoot = PossibleItems()
    }
}
```

Zaletą tego rozwiązania jest zawarcie całości kodu pliku w jednej klasie, zmienia się jednak sposób dostępu do stałej, np. podczas tworzenia obiektu klasy `EnemyBatCorpse` w klasie `EnemyBat`.

wyciag 1.14. `tworzenie obiektów klasy EnemyBatCorpse`

```
// wersja 1
import com.efm.entities.enemies.bat.EnemyBatCorpse
import com.efm.entities.enemies.bat.defaultLoot

val corpse = EnemyBatCorpse(loot = defaultLoot)

// wersja 2
import com.efm.entities.enemies.bat.EnemyBatCorpse

val corpse = EnemyBatCorpse(loot = EnemyBatCorpse.defaultLoot)
```

Ostatecznie zdecydowano, że stała przechowująca domyślny łup może być zastąpiona przez zmienną określającą łup w danej chwili. Będzie używana tylko podczas tworzenia zwłok, w klasie opisującej wroga (w tym przypadku w `EnemyBat`), dlatego zostanie przeniesiona do niej. Zmienna ta powinna stać się polem klasy, jeżeli jej wartość miałaby się różnić między obiektami, lub być zawarta w towarzyszącym obiekcie, jeśli ma być współdzielona przez wszystkie instancje. Tworzenie obiektów klasy `EnemyBatCorpse` będzie więc odpowiedzialnością metody `EnemyBat.getCorpse()`.

wyciag 1.15. `metoda getCorpse() w klasie EnemyBat`

```
override fun getCorpse() : EnemyCorpse =
    EnemyBatCorpse(this.position, this.loot)
```

1.4. Klonowanie obiektów

1.4.1. Motywacja

Występujący w programie interfejs `Item` reprezentuje pojedynczy przedmiot lub zbiór przedmiotów tego samego rodzaju. Żeby dzielić zbiory przedmiotów, a także wygodnie przenosić przedmioty i tworzyć je w różnych sytuacjach, zaprojektowano sposób klonowania obiektów tego typu. Wymaga on, żeby `Item` implementował interfejs `Cloneable`, udostępniając metodę `clone()`. Pozwala ona na kopiowanie pewnego przedmiotu bez znajomości jego typu (bezpośredniej nadklasy) w miejscu wywołania.

wyciag 1.16. `Item.kt`

```
interface Item : Cloneable
{
    val name : String
    val baseAPUseCost : Int

    fun getTexture() : Texture?
    fun selected()
    fun confirmed()

    public override fun clone() : Item
}
```

1.4.2. Cloneable

Przedmioty dzielą się na jednorazowego użytku, które łączą się w zbiory, i wielokrotnego użytku, które są przechowywane pojedynczo. W przykładzie zaprezentowany zostanie wyłącznie drugi rodzaj. Przedmioty przechowywane pojedynczo implementują interfejs `MultiUseMapItem`, dziedziczący po `Item`. Jedyne pole klasy o zmiennej wartości to `durability`, przechowujące informację o aktualnym stopniu wytrzymałości przedmiotu.

wyciag 1.17. `MultiUseMapItem.kt`

```
interface MultiUseMapItem : Item
{
    var durability : Int
    val maxDurability : Int
    val durabilityUseCost : Int
    ...
    override fun clone() : MultiUseMapItem
    {
        val copy = Class.forName(this::class.java.canonicalName)
                .getConstructor()
                .newInstance() as MultiUseMapItem
        copy.durability = this.durability
        return copy
    }
}
```

Dzięki realizacji metody `clone()` w interfejsie `MultiUseMapItem`, jej zachowanie jest jednakowe w każdej podklasie, a jej kod jest zawarty w jednym miejscu. Konstruktor odpowiedni dla danej podklasy jest znajdowany na podstawie jej nazwy. Po jego użyciu ustala się wartość zmiennej. Jedyne wymaganie, jakie muszą spełniać klasy implementujące interfejs, to posiadanie bezparametrowego konstruktora.

1.4.3. Data class

Przedstawione rozwiązanie opiera się na wykorzystaniu API `Reflections` języka Java, dzięki któremu możliwe jest poznanie nazwy klasy podczas wykonywania programu, nawet jeśli nie jest ona znana w trakcie kompilacji. Żeby rozwiązać pierwotny problem, ograniczając się do funkcji języka Kotlin, oraz

uniknąć potencjalnych niebezpieczeństw związanych z używaniem `Reflections`, kopia przedmiotu musi być tworzona poprzez jednoznaczne wywołanie konstruktora znanego na etapie kompilacji. Należy wtedy usunąć ciało metody `clone()` z interfejsu `MultiUseMapItem` i zaimplementować ją w każdej jego podklasie. Jeżeli nie są one abstrakcyjne i nie można po nich dziedziczyć, warto rozważyć deklarowanie ich jako `data class`. Jest to specjalny rodzaj klasy zaprojektowany w języku Kotlin, który automatycznie generuje podstawowe metody służące do wypisywania, porównywania oraz kopiowania obiektów. Co istotne, brane pod uwagę są tylko pola klasy będące jednocześnie parametrami konstruktora. W przypadku klas implementujących interfejs `MultiUseMapItem` należy więc umieścić zmienną `durability` w konstruktorze, a wynik domyślnie dostępnej metody `copy()` zwracać w nadpisywanej metodzie `clone()`.

wyciag 1.18. `Przykładowa klasa implementująca interfejs MultiUseMapItem jako data class`

```
data class Sledgehammer(override var durability : Int = 10) :
        MultiUseMapItem
{
    override val name : String = "Sledgehammer"
    override val baseAPUseCost : Int = 1
    override val maxDurability : Int = 10
    override val durabilityUseCost : Int = 1
    ...
    override fun clone() : Sledgehammer = this.copy()
}
```

Metoda `copy()` zwraca nowy obiekt, którego pola zadeklarowane w ciele klasy przyjmują wartości domyślne, a tym wymienionym w konstruktorze przypisywane są wartości aktualne oryginalnego obiektu. Inne automatycznie tworzone metody `data class` również używają jedynie pól deklarowanych w konstruktorze. Tylko one są uwzględniane w opisie klasy generowanym przez metodę `toString()` oraz porównywane w metodzie `equals()`. W Kotlinie zmienne przechowujące obiekty można porównywać poprzez użycie operatora `==`, który wykorzystuje metodę `equals()`, albo operatora `===`, który sprawdza, czy obie zmienne wskazują na ten sam obiekt w pamięci. W przypadku porównywania obiektów typu `data class` metodą `equals()` uznaje się je za równe wtedy i tylko wtedy, gdy każde z pól zdeklarowanych w konstruktorze przyjmuje

równą wartość w obu obiektach. Zachowanie to spełnia założenia postawione reprezentacji przedmiotów w projekcie.

1.5. Zarządzanie wartościami pustymi

1.5.1. Null

Typy danych wykorzystywane przez JVM dzielą się na proste (ang. *primitive types*) i referencyjne (ang. *reference types*), wskazujące na obiekty klas (por. rysunek 1.1). Jeżeli zmienna typu referencyjnego (referencja) nie wskazuje na żaden obiekt, przyjmuje się, że ma specjalną wartość pustą `null`. Próba interpretacji i użycia null jako obiektu jakiekolwiek klasy skutkuje wywołaniem błędu `NullPointerException`. Jako że domyślnie w języku Java wszystkie zmienne przechowujące obiekty mogą przyjmować wartość null, błąd ten występuje dość powszechnie. Projektując język Kotlin, starano się ograniczyć możliwość jego przypadkowego powodowania i związanych z tym zagrożeń. W Kotlinie nie używa się typów prostych, a wszystkie zmienne przechowujące obiekty domyślnie nie mogą przyjmować wartości null. Występujące w kodzie wyrażenie mogące przypisać null do zmiennej zostanie wykryte podczas kompilacji programu, a nowoczesne zintegrowane środowiska programistyczne potrafią je wskazać podczas pisania kodu źródłowego. Nie może zostać niezauważone w przeciwieństwie do błędu `NullPointerException`, który ujawnia się dopiero podczas wykonywania programu. Żeby dopuścić do przypisywania null jako wartości, przy deklaracji zmiennej należy dopisać symbol znaku zapytania (`?`) do nazwy typu. Tak tworzoną zmienną będzie się nazywać nullowalną (ang. *nullable*), w przeciwieństwie do zmiennej nienullowalnej (ang. *non-nullable*).

1.5.2. Bezpieczne używanie zmiennych nullowalnych

Jak wcześniej wyjaśniono, użycie zmiennej mogącej być null-em, może wywołać `NullPointerException`, więc kompilator Kotlina na to nie pozwala. Żeby bezpiecznie używać zmiennej nullowalnej, należy uprzednio sprawdzić, czy nie jest równa null, co niejawnie rzutuje typ zmiennej na typ nienullowalny, lub wykorzystać bezpieczne wywołanie ze znakiem zapytania. Metoda wywołana przez takie wyrażenie zostanie wykonana tylko wtedy, gdy zmienna wskazuje na obiekt. W przeciwnym razie wyrażenie przyjmie wartość null. W poniższym przykładzie jeżeli wyrażenie po lewej stronie operatora `?:` będzie równe null, zostanie zastąpione wartością po prawej stronie - pustą listą. Dlatego zmienna,

do której jest ono przypisywane, jest nienullowalna i można jej używać w standardowy sposób.

wyciag 1.19. `Ustalanie listy przeciwników, którzy będą patrolować pomieszczenie`

```
val roamingEnemies : List<Enemy> = World.currentRoom
        ?.getEnemies()
        ?.filter { Random.nextFloat() < it.roamingChance }
        ?: emptyList()
...
val enemyIterator = roamingEnemies.iterator()
```

1.6. Gotowe typy danych i funkcje wykorzystywane w projekcie

1.6.1. Kolekcje

Biblioteka standardowa Kotlina zawiera bogate w funkcjonalności implementacje podstawowych struktur danych - zbiorów, list i słowników, które zbiorczo nazywane są kolekcjami (ang. *collections*). Kolekcje udostępniają metody pozwalające jedynie na odczyt zapisanych w nich danych, ale każda z nich ma swój odpowiednik podlegający modyfikacji (ang. *mutable collections*). W projekcie *Escape from Morasko*, jeżeli było to możliwe, starano się korzystać z niezmiennych kolekcji, aby uniknąć błędów powodowanych przypadkową lub nieprzewidzianą ich modyfikacją. Najpowszechniej używanymi kolekcjami były listy. Na wyciągu 1.19 przedstawione zostało zastosowanie funkcji `filter()`, która zwraca listę zawierającą wyłącznie elementy spełniające zadany warunek, a także funkcji `emptyList()`. Wykorzystanie jej zamiast tworzenia pustej listy jest efektywniejsze obliczeniowo, ponieważ zamiast tworzyć nową kolekcję, zwraca ona już istniejącą, realizowaną jako `object`. Inne gotowe funkcje, które okazały się przydatne w projekcie służyły m.in. do wydzielania fragmentu listy, wybierania losowych elementów, ustalania losowej ich kolejności lub sortowania. Jako że stanowią część języka Kotlin, ich optymalizację i poprawność działania zapewniają jego twórcy, są dobrze opisane w dokumentacji, a ich używanie nie wymaga importowania dodatkowych bibliotek.

1.6.2. Funkcje rozszerzające

Składnia języka Kotlin pozwala na wygodne dodawanie funkcjonalności do istniejących klas bez ich modyfikacji. Służy do tego mechanizm tworzenia funkcji rozszerzających (ang. *extension functions*). Napisana na potrzeby projektu funkcja `largerMultipleOf()` może być używana na obiektach klasy `Int` jak jej metoda, mimo że jest deklarowana bezpośrednio w pliku z kodem źródłowym. Dzięki możliwości łączenia wywołań funkcji, kod jest zwięzły i czytelny.

wyciag 1.20. `Funkcja rozszerzająca klasę Int`

```
/** Returns the smallest multiple of x
 * that is larger than this value. */
fun Int.largerMultipleOf(x : Int) : Int
{
    val xAbs = abs(x)
    return when
    {
        xAbs == 0 -> 0
        this >= 0 -> this + (xAbs - this % xAbs)
        else      -> this + xAbs - (xAbs + this % xAbs) % xAbs
    }
}
...
maxItems = drawnItems.size.largerMultipleOf(5).coerceAtMost(25)
```

1.7. Podsumowanie

1.7.1. Korzyści zastosowania języka Kotlin

Do zrealizowania projektu *Escape from Morasko* zastosowano język Kotlin zamiast języka Java, ponieważ na podstawie wcześniejszych doświadczeń zespołu z oboma językami dostrzeżono liczne korzyści tego wyboru. Kotlin jest nowoczesny i aktywnie rozwijany przez profesjonalny zespół twórców jako projekt otwartoźródłowy. Oznacza to, że jest jawny, bezpieczny i dobrze udokumentowany. Jest już obecnie szeroko stosowany i przewiduje się, że jego popularność może dalej wzrastać. Jego biblioteka standardowa jest bogata w różnorodne, intu-

icyjnie nazwane funkcjonalności. Składnia języka nie wymaga kończenia linii symbolem średnika. Kotlin, podobnie jak Java, jest językiem obiektowym, ale w porównaniu do niej zapis najczęściej pisanych fragmentów kodu (np. metod zwracających lub przypisujących wartości pól klasy) jest krótszy i wygodniejszy w tworzeniu albo nawet generowany automatycznie. W uproszczeniu procesu pisania bezpiecznego kodu pomaga również obecność gotowych rozwiązań takich jak `data class` i `object`. Korzystanie z materiałów dotyczących języka Java jest ułatwiane dzięki narzędziom tłumaczącym kod źródłowy Javy na kod Kotlina. Możliwe jest nawet używanie obu języków w jednym projekcie. Kotlin jest też przystosowany do stosowania elementów paradygmatu programowania funkcyjnego, którego założenia, takie jak korzystanie z niezmiennych struktur danych, poprawiają jakość kodu.

Nie odnotowano żadnych problemów wynikających ze specyfiki języka Kotlin ani z faktu wykorzystania go zamiast Javy. Jego integracja z libGDX i testowanie z pomocą JUnit 4 również nie sprawiło trudności, mimo że te narzędzia są przeznaczone głównie dla języka Java. Dzięki zastosowaniu Kotlina kod projektu *Escape from Morasko* zawiera mniej linii kodu, jest bardziej zwięzły i czytelny, a zarazem możliwie bezpieczny, przy zachowaniu akceptowalnego poziomu efektywności obliczeniowej i czasu wykonania.

Bibliografia

[1] *Kotlin docs*
https://kotlinlang.org/docs [16.11.2023]

[2] *Using libGDX With Other JVM languages*
https://libgdx.com/wiki/jvm-langs/using-libgdx-with-other-jvm-languages [16.11.2023]

[3] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, D. Smith, *The Java® Virtual Machine Specification Java SE 21 Edition*, 2023-08-23
https://docs.oracle.com/javase/specs/jvms/se21/html/index.html [17.11.2023]

[4] *IBM SDK, Java Technology Edition 8, J9 VM reference, The JIT compiler*
https://www.ibm.com/docs/en/sdk-java-technology/8?topic=reference-jit-compiler [5.12.2023]

[5] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, G. Bierman, *The Java® Language Specification Java SE 21 Edition*, 2023-08-23
https://docs.oracle.com/javase/specs/jls/se21/html/index.html [30.12.2023]

[6] S. Atkinson, *Always Start With Eager Initialization*, 2016-03-29
https://dzone.com/articles/always-start-with-eager-initialisation [30.12.2023]